Data Extraction in ETL

Question 1: Describe different types of data sources used in ETL with suitable examples.

ETL stands for **Extract, Transform, and Load**. In an ETL process, data can be extracted from different types of sources:

1. **Relational databases:** These store structured data in tables made up of rows and columns. Examples include MySQL, PostgreSQL, and Microsoft SQL Server. A company might extract customer and order records from its database.
2. **Files:** Data may be stored in CSV, Excel, JSON, or XML files. For example, an ETL process might collect monthly sales data from CSV files or inventory data from an Excel workbook.
3. **APIs:** An API allows one software application to request data from another. For example, a business might use a weather API to collect current weather information or a payment API to retrieve transaction data.
4. **NoSQL databases:** These databases store data in formats such as documents or key-value pairs. Examples include MongoDB and Cassandra. An ETL process might extract product information stored as documents in MongoDB.
5. **Logs and event streams:** Applications and devices generate logs and events, such as website visits, user clicks, or system activity. These can be collected for reporting and analysis.
6. **Cloud storage:** Files stored in services such as Amazon S3 or Azure Blob Storage can be used as ETL sources. For example, a pipeline might process daily sales files uploaded to cloud storage.

The choice of data source depends on the data format, volume, update frequency, and the requirements of the ETL process.

Question 2: What is data extraction? Explain its role in the ETL pipeline.

Data extraction is the process of collecting data from one or more source systems and making it available for further processing. Sources may include databases, files, APIs, or cloud storage.

Extraction is the first stage of the ETL pipeline. It gathers the required data and passes it to the transformation stage, where the data is cleaned and prepared. The prepared data is then loaded into a destination, such as a data warehouse.

Data can be extracted as a full set or incrementally. A full extraction collects all the required data, while an incremental extraction collects only new or changed data. A reliable extraction process helps ensure that data is complete, accurate, and collected without unnecessarily affecting the source system.

Question 3: Explain the difference between CSV and Excel in terms of extraction and ETL usage.

CSV and Excel are both commonly used as data sources in ETL, but they differ in structure and processing.

A **CSV file** is a plain-text file that stores tabular data. Each line usually represents a row, and values are separated by commas or another delimiter. CSV files are simple, lightweight, and widely supported by ETL tools. They are often used to exchange data between systems. A CSV file typically contains one table and does not preserve formatting or formulas.

An **Excel file** is a workbook that can contain multiple worksheets, formulas, formatting, and other features. It is useful when data is organized across several sheets or prepared for people to review. During extraction, the ETL process may need to identify the correct workbook, worksheet, and data range. Formulas and formatting can also affect how values are read.

CSV is often a simple choice for automated data exchange, while Excel is useful for workbooks that contain multiple sheets or rely on spreadsheet features.

Question 4: Explain the steps involved in extracting data from a relational database.

The steps involved in extracting data from a relational database are:

1. **Identify the required data:** Determine which database, tables, columns, and records are needed.
2. **Establish a connection:** Configure a suitable database connector and provide the required credentials and permissions.

3. **Choose the extraction method:** Decide whether to extract all required records or only new and changed records.
4. **Retrieve the data:** Use an appropriate query or extraction tool to select the required records and fields.
5. **Handle large datasets:** If the dataset is large, extract it in batches to reduce memory use and avoid placing excessive load on the source database.
6. **Stage the extracted data:** Store the extracted data temporarily so it can be checked and passed to the transformation stage.
7. **Validate the results:** Check record counts, required fields, and data completeness. Record any errors so they can be investigated.
8. **Maintain extraction records:** Save information such as the extraction time and progress markers to support monitoring and recovery if the process fails.

Question 5: Explain three common challenges faced during data extraction.

Three common challenges faced during data extraction are:

1. **Poor data quality:** Source data may contain missing values, duplicates, invalid records, or inconsistent formats. These problems can affect the accuracy of later analysis. Validation helps identify such issues.
2. **Large data volumes:** Extracting a large amount of data can take a long time, use significant memory, and affect the performance of the source system. Batch processing and incremental extraction can help manage the workload.
3. **Unreliable connections or changing sources:** A connection may fail during extraction, or the source structure may change. These issues can interrupt the process or cause data to be missed. Monitoring, retry mechanisms, and checks for changes help make extraction more reliable.

Question 6: What are APIs? Explain how APIs help in real-time data extraction.

An **API**, or Application Programming Interface, is a set of rules that allows one software application to communicate with another. An API can provide access to data through requests and responses, often using formats such as JSON.

APIs help with real-time or near-real-time data extraction by allowing an ETL system to request current data from a source application. Some APIs also send notifications when

new data or events are available. This lets a pipeline collect updates more frequently than a process that relies only on occasional file exports.

For example, an ETL process could use a payment service API to retrieve new transactions throughout the day. The frequency and amount of data available depend on the API and any limits set by the service.

Question 7: Why are databases preferred for enterprise-level data extraction?

Databases are often preferred for enterprise-level data extraction because they can store and manage large amounts of structured data. Their data is organized in tables, making it easier to select specific records and fields.

Databases also provide access controls to help protect sensitive information and support multiple users and applications. They can offer reliable data management, backup and recovery features, and tools for monitoring performance. These capabilities help organizations extract data consistently and securely for reporting and analysis.

Question 8: What steps should an ETL developer take when extracting data from large CSV files (1 GB or more)?

When extracting data from a large CSV file, an ETL developer should take the following steps:

1. **Inspect the file:** Check the file's encoding, delimiter, header row, column names, and data format before processing it.
2. **Use chunk-based processing:** Read and process a limited number of rows at a time instead of loading the entire file into memory.
3. **Define data types carefully:** Specify how values such as dates, numbers, and empty fields should be interpreted to avoid incorrect results.
4. **Validate the records:** Check for missing fields, malformed rows, unexpected values, and duplicate records.
5. **Record errors:** Save invalid records and error details separately so they can be reviewed without stopping the entire process unnecessarily.
6. **Make the extraction recoverable:** Track progress so processing can resume after a failure without repeating completed work or missing records.

7. **Monitor the process:** Track processing time, memory use, record counts, and errors.
8. **Verify the output:** Compare the number of extracted records with the expected number and confirm that the data was transferred successfully to the next stage of the ETL pipeline.